Data Analytics (DA) Module-End -4

Module End Assignment: Python

“Customer Support Ticket Analyzer”

Problem Statement

Customer support teams handle numerous service tickets daily. Analysing support tickets helps identify common issues, customer sentiment, support quality, and areas for improvement.

In this project, you will build a **Python-based Ticket Analysis System** that stores, cleans, analyses, and extracts insights from customer support tickets.

Step 1: Preloaded Tickets



```
# Step 1 - preloaded tickets
ticket_data = {
    'Ticket_No': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Customer_Name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh',
                      'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha'],
    'Issue_Description': [
        ' Internet not working!!! ', 'slow response, very poor service ',
        'GREAT support! issue resolved.', ' okay... need help ',
        'not BAD but slow', 'Excellent guidance, Very Helpful!',
        'good support and good behaviour!', 'Poor handling of technical issue',
        'Satisfied. Could be better.', 'Good service... quick response.'],
    'Priority': ['High', 'Low', 'High', 'Medium', 'Low',
                 'High', 'Medium', 'High', 'Low', 'Medium']
}
print(ticket_data)
```

```
{'Ticket_No': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 'Customer_Name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha'], 'Issue_Descript
```



```
print("INITIAL TICKETS")
print("-" * 70)
for i in range(len(ticket_data['Ticket_No'])):
    print(f"Ticket {ticket_data['Ticket_No'][i]:>2} | "
          f"{ticket_data['Customer_Name'][i]:<7} | "
          f"{ticket_data['Priority'][i]:<6} | "
          f"{ticket_data['Issue_Description'][i]!r}")
```

```
INITIAL TICKETS
----------------------------------------------------------------------
Ticket  1 | Ravi    | High   | ' Internet not working!!! '
Ticket  2 | Meera   | Low    | 'slow response, very poor service '
Ticket  3 | Sam     | High   | 'GREAT support! issue resolved.'
Ticket  4 | Anu     | Medium | ' okay... need help '
Ticket  5 | Rakesh  | Low    | 'not BAD but slow'
Ticket  6 | Divya   | High   | 'Excellent guidance, Very Helpful!'
Ticket  7 | Arjun   | Medium | 'good support and good behaviour!'
Ticket  8 | Kiran   | High   | 'Poor handling of technical issue'
Ticket  9 | Leela   | Low    | 'Satisfied. Could be better.'
Ticket 10 | Nisha   | Medium | 'Good service... quick response.'
```

Step 2: Add More Tickets

```python
# Step 2 - add more tickets
how_many = int(input("How many new tickets do you want to add? "))

for _ in range(how_many):
    name = input("Enter customer name: ").strip()
    issue = input("Enter issue description: ")

    while True:                                  # keep asking until valid
        priority = input("Enter priority (High / Medium / Low): ").strip().capitalize()
        if priority in ("High", "Medium", "Low"):
            break
        print("Invalid priority. Please type High, Medium or Low.")

    new_no = ticket_data['Ticket_No'][-1] + 1     # last number + 1
    ticket_data['Ticket_No'].append(new_no)
    ticket_data['Customer_Name'].append(name)
    ticket_data['Issue_Description'].append(issue)
    ticket_data['Priority'].append(priority)
    print(f"Ticket {new_no} added.")

print("Total tickets now:", len(ticket_data['Ticket_No']))
```

```
How many new tickets do you want to add? 2
```

```
How many new tickets do you want to add? 2
Enter customer name: priya
Enter issue description: Router keeps disconnecting every hour and nobody called me back!
Enter priority (High / Medium / Low): urgent
Invalid priority. Please type High, Medium or Low.
Enter priority (High / Medium / Low): high
Ticket 11 added.
Enter customer name: vikram
Enter issue description: ok, quick reply from agent
Enter priority (High / Medium / Low): medium
Ticket 12 added.
Total tickets now: 12
```

Step 3: Text Cleaning for Issue Descriptions

[4] ✓ 0s

```
text = " Internet not working!!! "
print("Original      :", repr(text))

text = text.lower()
print("Lowercase     :", repr(text))

for ch in ".,!?-":
    text = text.replace(ch, " ")
print("No punctuation:", repr(text))

words = text.split()
print("Split         :", words)

text = " ".join(words)
print("Joined        :", repr(text))
```

```
Original      : ' Internet not working!!! '
Lowercase     : ' internet not working!!! '
No punctuation: ' internet not working    '
Split         : ['internet', 'not', 'working']
Joined        : 'internet not working'
```

[5] ✓ 0s

```
print("ok".replace("ok", "okay"))
print("book".replace("ok", "okay"))
```

```
okay
bookay
```

[6] ✓ 0s

```
# Step 3 - text cleaning
slang = {"ok": "okay", "pls": "please", "plz": "please", "thx": "thanks", "u": "you"}

def clean_text(text):
    text = text.lower()                     # lowercase
    for ch in ".,!?-":                      # remove punctuation
        text = text.replace(ch, " ")
    words = text.split()                    # fixes extra spaces and trims the ends
    fixed = []
    for word in words:
        fixed.append(slang.get(word, word))  # swap slang, keep other words
    return " ".join(fixed)

cleaned = []
for text in ticket_data['Issue_Description']:
    cleaned.append(clean_text(text))
ticket_data['Issue_Description'] = cleaned
```

Step 4: Keyword-Based Issue Insights

[10]
✓ 0s

```
# Step 4 - keyword search function
def count_tickets_with_word(word):
    word = word.lower()
    count = 0
    for description in ticket_data['Issue_Description']:
        if word in description.lower().split():
            count += 1
    return count
print('Tickets containing "poor"     :', count_tickets_with_word("poor"))
print('Tickets containing "good"     :', count_tickets_with_word("good"))
print('Tickets containing "slow"     :', count_tickets_with_word("slow"))
print('Tickets containing "excellent":', count_tickets_with_word("excellent"))
```

```
Tickets containing "poor"     : 2
Tickets containing "good"     : 2
Tickets containing "slow"     : 2
Tickets containing "excellent": 1
```

[11]
✓ 0s

```
print(count_tickets_with_word("POOR"), count_tickets_with_word("Poor"), count_tickets_with_word("poor"))
```

```
2 2 2
```

Step 5: Final Summary & Insights

Produce detailed analytics:

Display Final Cleaned ticket_dataicely formatted dictionary-of-lists output

```
[12]
✓ 0s
# Step 5.1 - final cleaned data
from pprint import pprint
pprint(ticket_data, sort_dicts=False)
```

```
{'Ticket_No': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
 'Customer_Name': ['Ravi',
                   'Meera',
                   'Sam',
                   'Anu',
                   'Rakesh',
                   'Divya',
                   'Arjun',
                   'Kiran',
                   'Leela',
                   'Nisha',
                   'priya',
                   'vikram'],
 'Issue_Description': ['internet not working',
                       'slow response very poor service',
                       'great support issue resolved',
                       'okay need help',
                       'not bad but slow',
                       'excellent guidance very helpful',
                       'good support and good behaviour',
                       'poor handling of technical issue',
                       'satisfied could be better',
                       'good service quick response',
                       'router keeps disconnecting every hour and nobody '
                       'called me back',
                       'okay quick reply from agent'],
 'Priority': ['High',
              'Low',
              'High',
              'Medium',
              'Low',
              'High',
              'Medium',
              'High',
              'Low',
              'Medium',
              'High',
              'Medium']}
```

2. Priority Analysis

Compute:

- Number of **High-priority** tickets

- Number of **Medium priority** tickets
- Number of **Low-Priority** tickets

```
[13]
✓ 0s
# Step 5.2 - priority analysis
print("High priority tickets  :", ticket_data['Priority'].count('High'))
print("Medium priority tickets:", ticket_data['Priority'].count('Medium'))
print("Low priority tickets   :", ticket_data['Priority'].count('Low'))

High priority tickets  : 5
Medium priority tickets: 4
Low priority tickets   : 3
```

3. Find the Ticket With the Longest Issue Description

Based on **word count**, print:

- Ticket number
- Customer name
- Cleaned issue text
- Word count

```
[14]
✓ 0s
# Step 5.3 - ticket with the longest description
longest_index = 0
longest_count = 0
for i in range(len(ticket_data['Ticket_No'])):
    word_count = len(ticket_data['Issue_Description'][i].split())
    if word_count > longest_count:
        longest_count = word_count
        longest_index = i

print("Ticket number :", ticket_data['Ticket_No'][longest_index])
print("Customer name :", ticket_data['Customer_Name'][longest_index])
print("Issue text    :", ticket_data['Issue_Description'][longest_index])
print("Word count    :", longest_count)

Ticket number : 11
Customer name : priya
Issue text    : router keeps disconnecting every hour and nobody called me back
Word count    : 10
```

4. Extract Unique Words Used

Generate a **set** of all unique words across all issue descriptions.

Show:

- Count of unique words

The word list (sorterd



```
[15]
✓ 0s
# Step 5.4 - unique words
unique_words = set()
for description in ticket_data['Issue_Description']:
    for word in description.split():
        unique_words.add(word)

print("Count of unique words:", len(unique_words))
print("Word list (sorted)   :", ", ".join(sorted(unique_words)))

Count of unique words: 43
Word list (sorted)   : agent, and, back, bad, be, behaviour, better, but, called, could, disconnecting, every, excellent, from, good, great, guidance, handling, help,
```

One-Page Summary and Sharing

Title and objective	Customer Support Ticket Analysis. One sentence: analyse support tickets to find common issues, sentiment and priorities.
Data	12 tickets: 10 preloaded and 2 added by the user. Fields: ticket number, customer, issue description, priority.
Method	Cleaned text (lowercase, punctuation removed, extra spaces fixed, slang replaced); counted keywords with a function; analysed priority; found the longest description; extracted unique words.
Priority findings	High: 5, Medium: 4, Low: 3. Comment on which level is most common and what that means for staffing.
Keyword findings	poor: 2, good: 2, slow: 2, excellent: 1. Positive words (good, excellent) vs negative or problem words (poor, slow).
Longest ticket	Ticket 11 (Priya) with 10 words: the most detailed complaint.
Vocabulary	43 unique words across all descriptions.
Recommendations	2 or 3 actions based on your numbers, for example: speed up response times (slow, poor), resolve High priority tickets first, keep the practices behind positive feedback.
Limitations	Only a few tickets; keyword counting does not understand context (for example, "not bad" is positive but contains the word bad).